HashDit

Smart Contract Security

ListaDao Broker Batch Ops

Security Audit Report

ListaDao

May 21 2026 – May 26 2027

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	21 May 2026 — 26 May 2026
Overall Risk [Optional]	**Informational**
Project Name	ListaDao
Github	https://github.com/lista-dao/moolah/pull/180
Commit	feat/broker-batch-ops

Smart Contract List

No.	Contract Name	Verdict	Details
1	BatchManagementUtils.sol	Informational	[I01] [I02] [I03] [I04]

Findings

[I01] batchUpdateFixedTermAndRate cannot drive CreditBroker due to ABI, role, and APR bound mismatches	
Contract	BatchManagementUtils.sol
Severity	Informational
Description	**Description** The BatchManagementUtils.batchUpdateFixedTermAndRate function is currently only intended to be batchable across LendingBroker. For CreditBroker, the current implementation makes this impossible due to three distinct mismatches. 1. Struct and Selector Mismatch: BatchManagementUtils imports FixedTermAndRate from IBroker.sol, which defines a 3 field struct { uint256 termId; uint256 duration; uint256 apr; }. CreditBroker, however, uses a different FixedTermAndRate from ICreditBroker.sol containing 4 fields { uint256 termId; uint256 duration; uint256 apr; FixedTermType termType; }. This produces two different canonical ABI signatures: updateFixedTermAndRate((uint256,uint256,uint256),bool) for LendingBroker and updateFixedTermAndRate((uint256,uint256,uint256,uint8),bool) for CreditBroker, with different 4 byte selectors and different calldata lengths (96 bytes vs 128 bytes for the tuple). A call dispatched from BatchManagementUtils will not match CreditBroker's selector and will revert since no fallback exists on CreditBroker. 2. Role Mismatch: LendingBroker.updateFixedTermAndRate is restricted by onlyRole(BOT), while CreditBroker.updateFixedTermAndRate is restricted by onlyRole(MANAGER). BatchManagementUtils only validates that the caller holds the BOT role, so even if the ABI issue were fixed, calls to CreditBroker would fail authorization because msg.sender would need MANAGER, not BOT. 3. APR Bound Mismatch: LendingBroker caps APR at 30% and floors it at 1%, while CreditBroker via CreditBrokerLib.MAX_FIXED_TERM_APR = 9e27 permits APR up to roughly 800%. Passing the same terms[i] to both brokers would either be rejected by LendingBroker's narrower band or be unable to express the high APR products that CreditBroker supports.
Recommendation	Consider a separate function to batch update fixedTermandRate function in the future if there are multiple CreditBrokers in the future e.g. differing LOAN_TOKEN. If not, including sufficient documentation highlighting a single current CreditBroker market is acceptable.
Status	Waiting for confirmation from the development team.

[I02] Batch helper functions emit no events for off-chain tracking

Contract	BatchManagementUtils.sol
Severity	Informational
Description	**Description** The batch update and setter functions in BatchManagementUtils.sol do not emit any events of their own. Off-chain monitoring systems can only observe the underlying broker events triggered by each inner call, which leaves no way to correlate those events back to the specific batch transaction that produced them.
Recommendation	Optionally emit a dedicated event from each batch function, such asBatchExecuted() on every call. If not, no fix is required as well if reliance of underlying events is accepted
Status	Waiting for confirmation from the development team.

[I03] SetWhitelist event declared but never emitted	
Contract	BatchManagementUtils.sol
Severity	Informational
Description	**Description** The SetWhitelist event in BatchManagementUtils.sol is declared in the contract but never emitted anywhere in the code. This results in dead code that has no functional purpose and may mislead readers, integrators, and off-chain consumers into expecting whitelist updates to be observable on-chain, when in fact no such event is ever produced. JavaScript `event SetWhitelist(address indexed to, bool status);`
Recommendation	Either remove the SetWhitelist event declaration if it is not needed, or wire it up correctly by emitting it inside the function that performs the whitelist update so the change becomes observable on-chain
Status	Waiting for confirmation from the development team.

[I04] Unvalidated broker addresses allow forged authorization checks	
Contract	BatchManagementUtils.sol
Severity	Informational
Description	**Description** The batchUpdateFixedTermAndRate and batchRefinance functions in BatchManagementUtils accept a brokers array directly from calldata without any validation against an approved list. For each entry, the contract performs the

authorization check by calling the hasRole checks and dispatches the action to the same brokers[i] address. Because both the role check and the call target come from the same unvalidated input, the access control gate can be trivially bypassed by any caller.

An attacker can deploy a malicious contract whose hasRole(...) always returns true. Calling batchUpdateFixedTermAndRate() passes the role check because of its custom logic, and the contract then allow calls towards Evil.updateFixedTermAndRate() with msg.sender = address(BatchManagementUtils).

Note:

- This does not produce a privilege access bypass against the real brokers. From inside a malicious contract (e.g. Evil), any further call to a real LendingBroker would carry msg.sender = Evil, not BatchManagementUtils, so the real broker's BOT check would still fail.

- Does not apply to the batchSetMarketFee given the Moolah contract is a deployer set immutable address

As such, the impact is limited to spoofing transactions from the deployed BatchManagementUtils contract

JavaScript

```
/**
   * @dev Update fixed term and rate across multiple LendingBrokers in a
single transaction.
   * @param brokers The array of LendingBroker addresses.
   * @param terms The array of fixed term and rate schemes corresponding
to each broker.
   * @param removes The array of flags indicating whether to remove the
term (true) or add/update it (false).
   *
   * Requirements:
   * - Caller must have the BOT role on every broker in `brokers`.
   * - This contract must also have the BOT role on every broker in
`brokers` so it can forward the call.
   * - All three arrays must have the same length and be non-empty.
   */
  function batchUpdateFixedTermAndRate(
    address[] calldata brokers,
    FixedTermAndRate[] calldata terms,
    bool[] calldata removes
  ) external {
    require(
      brokers.length == terms.length && brokers.length == removes.length
&& brokers.length > 0,
      "Array length mismatch"
    );
```

```
    for (uint256 i = 0; i < brokers.length; i++) {
      require(IAccessControl(brokers[i]).hasRole(BOT, msg.sender), "Not
bot of broker");
      ILendingBrokerBot(brokers[i]).updateFixedTermAndRate(terms[i],
removes[i]);
    }
  }

  /**
   * @dev Refinance matured fixed positions for multiple (broker, user)
pairs in a single transaction.
   * @param brokers The array of LendingBroker addresses.
   * @param users The array of users whose matured fixed positions will
be refinanced.
   * @param posIds The array of posId arrays, one per (broker, user)
pair.
   *
   * Requirements:
   * - Caller must have the BOT role on every broker in `brokers`.
   * - This contract must also have the BOT role on every broker in
`brokers` so it can forward the call.
   * - All three arrays must have the same length and be non-empty.
   */
  function batchRefinance(address[] calldata brokers, address[] calldata
users, uint256[][] calldata posIds) external {
    require(
      brokers.length == users.length && brokers.length == posIds.length
&& brokers.length > 0,
      "Array length mismatch"
    );
    for (uint256 i = 0; i < brokers.length; i++) {
      require(IAccessControl(brokers[i]).hasRole(BOT, msg.sender), "Not
bot of broker");

ILendingBrokerBot(brokers[i]).refinanceMaturedFixedPositions(users[i],
posIds[i]);
    }
  }
```

Recommendation	Maintain an on-chain whitelist of approved LendingBroker contracts managed by an admin role, and require each brokers[i] to be present in this whitelist before performing the role check or dispatching the external call.
Status	Waiting for confirmation from the development team.